

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Trà Vinh, ngày tháng năm

Giáo viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

Trà Vinh, ngày tháng năm

Thành viên hội đồng
(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Kính thưa Thầy Ngô Thanh Huy, tôi xin gửi lời cảm ơn chân thành tới Thầy đã tận tình hướng dẫn và hỗ trợ tôi trong quá trình thực hiện đề tài "Phát triển ứng dụng minichat bằng Java" và hoàn thành báo cáo môn Ứng dụng phát triển hướng dịch vụ. Nhờ vào những hướng dẫn rõ ràng và các buổi học trực tuyến của Thầy, tôi đã hiểu rõ hơn nhiều khái niệm khó và biết cách áp dụng kỹ thuật lập trình vào thực tế.

Tôi rất trân trọng sự kiên nhẫn, nhiệt tình của Thầy khi giải đáp thắc mắc và góp ý để tôi hoàn thiện sản phẩm. Những kiến thức và cách tiếp cận vấn đề mà Thầy chia sẻ đã giúp tôi cải thiện cả kỹ năng chuyên môn lẫn tư duy lập trình.

Một lần nữa tôi chân thành cảm ơn Thầy và mong tiếp tục nhận được sự chỉ dẫn của Thầy trong những môn học tới.

Trân trọng cảm ơn,

MỤC LỤC

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	1
NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG	2
LỜI CẢM ƠN	3
MỤC LỤC	4
DANH MỤC HÌNH ẢNH	6
TÓM TẮT	7
MỞ ĐẦU	9
CHƯƠNG 1: TỔNG QUAN	11
1.1 Mô hình Client-Server và vai trò trong phát triển ứng dụng	11
1.2 Giao thức Socket và khả năng truyền tải dữ liệu	11
1.3 Mã hóa và bảo mật	11
1.4 Java và lợi ích trong việc phát triển hệ thống	11
1.5 Hướng phát triển và mở rộng trong tương lai	12
CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT	13
2.1. Mô hình Client-Server trong phát triển chat	13
2.1.1. Khái niệm và nguyên lý hoạt động của mô hình Client-Server	13
2.1.2. Ưu điểm và hạn chế khi áp dụng mô hình Client-Server trong chat	14
2.2. Lập trình Socket trong Java	15
2.2.1. Socket TCP và UDP trong Java	15
2.2.2. Cơ chế gửi và nhận dữ liệu qua Socket	16
2.3. Quản lý tin nhắn và kết nối	17
2.3.1. Quản lý kết nối Client-Server đa luồng	17
2.3.2. Lưu trữ và đồng bộ tin nhắn	18
2.4. Bảo mật trong ứng dụng Chat	19
2.4.1. Mã hóa dữ liệu truyền qua mạng	19
2.4.2. Xác thực người dùng và kiểm soát truy cập	21
2.5. Tạo và quản lý Nhóm	21
2.5.1. Tổ chức nhóm chat	21

2.5.2. Đồng bộ tin nhắn trong nhóm	22
2.5.3. Quản lý lịch sử và thông báo nhóm	23
CHƯƠNG 3: NỘI DUNG THỰC HIỆN	25
3.1. Giải pháp	25
3.2. Thực nghiệm (<i>phụ lục</i>)	26
3.3. Thực hành	26
CHƯƠNG 4: ĐÁNH GIÁ KẾT QUẢ	29
4.1. Kết quả đạt được	29
4.2. Hạn chế	29
4.3. Hướng phát triển	30
CHƯƠNG 5: KẾT LUẬN	31
5.1. Kết quả đạt được	31
5.2. Đóng góp mới	31
5.3. Đề xuất phát triển	32
5.4. Kết luận tổng thể	32
DANH MỤC TÀI LIỆU THAM KHẢO	34
PHỤ LỤC	35
* Code ChatServer.java	35
* Code ChatClient.java	38

DANH MỤC HÌNH ẢNH

Hình 1. Kiến trúc client-server của MiniChat	11
Hình 2. Mô hình giao tiếp client - server	14
Hình 3. Luồng xử lý của dự án	18
Hình 4. Mô phỏng giao diện sau khi kết nối và gửi tin nhắn	22

DANH MỤC BẢNG BIỂU

Bảng 1. Bảng kết quả đề tài	9
Bảng 2. Danh sách các file trong dự án	13
Bảng 3. Danh sách lớp của đề tài	20
Bảng 4. Đánh giá kết quả đạt được	25

TÓM TẮT

Đề tài MiniChat xây dựng một ứng dụng trò chuyện nội bộ đơn giản bằng ngôn ngữ Java. Ứng dụng vận hành theo mô hình client-server, trong đó server đóng vai trò trung tâm tiếp nhận kết nối, quản lý danh sách client và chuyển tiếp tin nhắn giữa các người dùng. Client có hai hình thức sử dụng: client dòng lệnh phục vụ kiểm thử nhanh và client giao diện đồ họa Java Swing phục vụ thao tác trực quan.

Về mặt kỹ thuật, đề tài tập trung vào các kiến thức nền tảng của lập trình mạng như TCP socket, ServerSocket, Socket, luồng vào ra văn bản, lập trình đa luồng và xử lý tài nguyên kết nối. Server lắng nghe tại cổng 9999. Khi một client kết nối, server tạo một đối tượng ClientHandler và chạy đối tượng này trong một Thread riêng. Nhờ đó, nhiều client có thể kết nối đồng thời mà không làm nghẽn luồng chính của server.

Về mặt giao diện, ChatClientGUI sử dụng JFrame, JTextArea, JTextField, JButton và JLabel. Người dùng nhập username, bấm Connect để kết nối đến server, nhập tin nhắn và bấm Send để gửi. Giao diện cũng hiển thị trạng thái kết nối và tự lọc các thông báo kỹ thuật không cần thiết như lời nhắc nhập username từ server.

Kết quả đạt được là một ứng dụng chat hoạt động được trong môi trường localhost, có thể kiểm thử với nhiều cửa sổ client. Đề tài tuy còn đơn giản nhưng phản ánh đầy đủ các bước quan trọng khi xây dựng một hệ thống truyền thông thời gian gần thực: mở cổng server, thiết lập kết nối client, truyền thông điệp, xử lý đồng thời và đóng kết nối an toàn.

Nội dung	Kết quả chính
Mô hình	Client-server chạy trên localhost qua TCP socket.
Server	Lắng nghe cổng 9999, chấp nhận nhiều client và broadcast tin nhắn.
Client	Có bản dòng lệnh và bản giao diện Swing.
Đa luồng	Mỗi client được xử lý bởi một Thread riêng.
Kết quả	Gửi nhận tin nhắn giữa nhiều cửa sổ client thành công.

Bảng 1. Bảng kết quả đề tài

MỞ ĐẦU

Lý do chọn đề tài:

Trong đời sống hiện nay, các ứng dụng nhắn tin là một dạng phần mềm rất quen thuộc. Người dùng có thể gửi thông điệp, nhận phản hồi, trao đổi thông tin nhóm và làm việc cộng tác qua mạng. Dù các sản phẩm thực tế như Messenger, Zalo, Slack hay Microsoft Teams có kiến trúc phức tạp, phần lõi của chúng vẫn bắt đầu từ nguyên lý cơ bản: một chương trình gửi dữ liệu qua mạng và một chương trình khác nhận dữ liệu đó.

Đề tài MiniChat được chọn vì có tính thực hành cao. Chỉ với một ứng dụng nhỏ, sinh viên có thể tiếp cận nhiều khái niệm quan trọng: socket, cổng mạng, luồng dữ liệu, server trung tâm, client, đa luồng, giao diện và xử lý lỗi. Đây là những kiến thức nền tảng cho các môn Lập trình mạng, Hệ phân tán và Phát triển ứng dụng.

Mục đích:

Mục tiêu đầu tiên là xây dựng được một server chat có khả năng lắng nghe kết nối từ nhiều client. Server phải chấp nhận client mới, tạo luồng xử lý riêng và chuyển tiếp tin nhắn đến các client còn lại.

Mục tiêu thứ hai là xây dựng client có khả năng kết nối đến server, gửi username, gửi nội dung chat và nhận nội dung từ người dùng khác. Ngoài client terminal, đề tài hướng đến giao diện dễ thao tác hơn bằng Java Swing.

Mục tiêu thứ ba là rèn luyện cách tổ chức mã nguồn Java theo nhiều lớp, mỗi lớp có trách nhiệm rõ ràng. Cách chia lớp giúp chương trình dễ đọc, dễ kiểm thử và dễ mở rộng trong tương lai.

Phạm vi đề tài:

Đề tài tập trung vào chat văn bản trong môi trường cục bộ localhost. Chương trình chưa triển khai kết nối qua Internet công cộng, chưa dùng cơ sở dữ liệu, chưa mã hóa dữ liệu và chưa có hệ thống tài khoản thật.

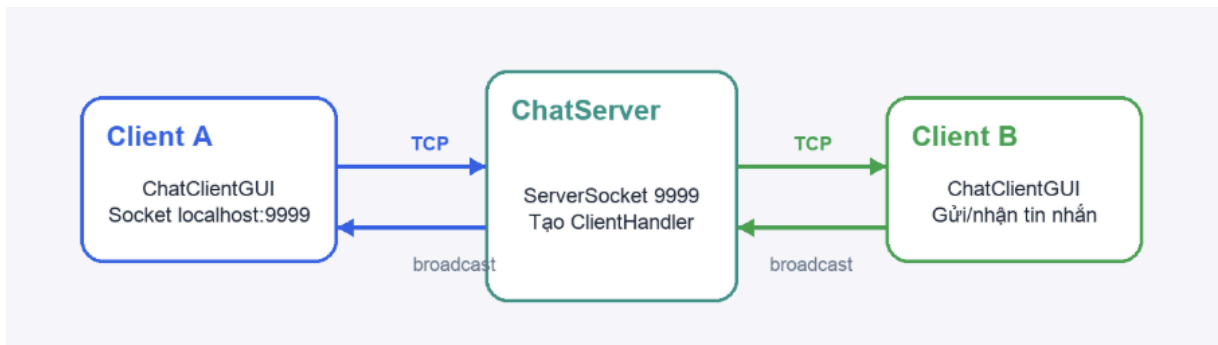
Các chức năng chính gồm: chạy server, kết nối nhiều client, nhập username, gửi tin nhắn, nhận tin nhắn, hiển thị trạng thái kết nối và thoát bằng lệnh /quit hoặc nút Disconnect.

Phạm vi nghiên cứu:

Phạm vi nghiên cứu trong đề tài này tập trung vào việc phát triển ứng dụng MiniChat cho nhóm người dùng nhỏ, với các tính năng cơ bản như kết nối, tham gia nhóm và gửi nhận tin nhắn. Hệ thống sẽ được triển khai trong môi trường thử nghiệm với một số lượng người dùng hạn chế. Phạm vi nghiên cứu không bao gồm việc phát triển giao diện người dùng phức tạp hay xử lý các vấn đề mở rộng quy mô lớn, nhưng sẽ tập trung vào việc tối ưu hóa hiệu suất và bảo mật cho các kết nối giữa client và server.

CHƯƠNG 1: TỔNG QUAN

MiniChat là ứng dụng mô phỏng một phòng chat chung. Người dùng mở client, nhập tên, kết nối đến server và gửi tin nhắn. Server nhận tin nhắn từ một client và chuyển tiếp đến các client khác. Vì server giữ vai trò trung gian, các client không cần kết nối trực tiếp với nhau.



Hình 1. Kiến trúc client-server của MiniChat

1.1 Các thành phần chính

Lớp/Tập	Chức năng	Ghi chú
chatserver.java	Mở ServerSocket tại cổng 9999, chấp nhận kết nối và quản lý danh sách client.	Thành phần trung tâm
clienthandler.java	Xử lý riêng từng client, đọc username, đọc tin nhắn và gọi broadcast.	Chạy trong Thread riêng.
chatclient.java	Client dòng lệnh, phù hợp kiểm thử nhanh.	Dùng terminal.
chatclientgui.java	Client giao diện Swing, có vùng chat và nút thao tác.	Dùng cho người dùng đầu cuối.
module-info.java	Khai báo module Minichat và quyền dùng java.desktop.	Cần cho Swing.
messageprotocol.java	Khai báo một số hằng lệnh dự kiến.	Có thể dùng khi mở rộng.
chatgroup.java	Mô hình nhóm chat đơn giản.	Có thể dùng khi phát triển chat nhóm.

Bảng 2. Danh sách các file trong dự án

1.2 Chức năng đã xây dựng

- Server lắng nghe kết nối tại localhost, cổng 9999.
- Client có thể nhập username và tham gia phòng chat.
- Server thông báo khi người dùng tham gia hoặc rời khỏi phòng chat.
- Client gửi tin nhắn; server chuyển tiếp tin nhắn đến các client khác.
- Giao diện Swing hiển thị trạng thái kết nối, vùng chat và ô nhập tin nhắn.
- Client xử lý lỗi khi server chưa chạy bằng thông báo dễ hiểu.

1.3 Ý nghĩa học tập

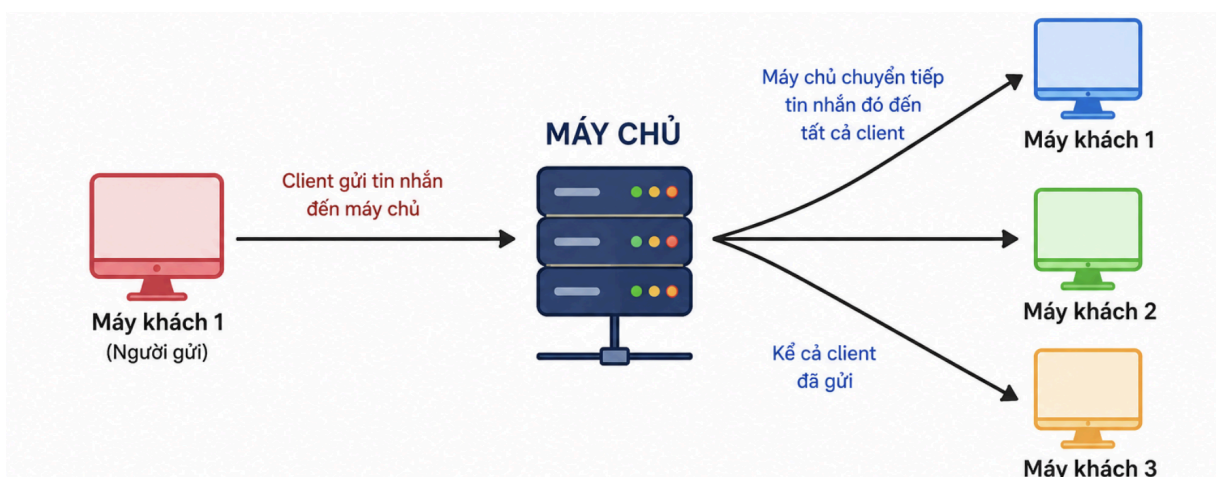
Đề tài giúp người học chuyển từ lý thuyết sang thực hành. Thay vì chỉ đọc về socket, người học tự tạo server, tự mở client, quan sát lỗi khi chưa chạy server và hiểu vì sao cần chạy server trước. Quá trình này tạo nền tảng tốt để phát triển các ứng dụng mạng lớn hơn.

CHƯƠNG 2: NGHIÊN CỨU LÝ THUYẾT

2.1. Mạng máy tính và truyền thông giữa tiến trình

Trong lập trình mạng, điều quan trọng không chỉ là hai máy tính giao tiếp với nhau mà còn là hai tiến trình phần mềm trao đổi dữ liệu. Một tiến trình có thể là server đang chạy nền, tiến trình còn lại là client do người dùng mở. Mỗi tiến trình cần biết địa chỉ và cổng để gửi dữ liệu đúng nơi.

Trong MiniChat, các tiến trình chạy trên cùng một máy nên địa chỉ là localhost. Dù chạy cùng máy, chương trình vẫn dùng cơ chế mạng thật của hệ điều hành. Điều này giúp mô hình có thể mở rộng sang nhiều máy tính khi thay localhost bằng địa chỉ IP phù hợp.



Hình 2. Mô hình giao tiếp client - server

Mô hình hoạt động cơ bản vận hành theo quy trình: khi một client gửi tin nhắn hoặc yêu cầu đến máy chủ (server), server tiếp nhận dữ liệu, thực hiện xử lý nghiệp vụ bao gồm kiểm tra thông tin người nhận, lưu trữ tin nhắn và/hoặc thực hiện các thao tác cần thiết khác, sau đó chuyển kết quả hoặc tin nhắn tới client đích. Cơ chế này đảm bảo quản lý tập trung dữ liệu, giảm thiểu nguy cơ mất mát thông tin và tạo điều kiện thuận lợi cho việc đồng bộ tin nhắn giữa các client.

Trong các ứng dụng chat hiện đại, server thường được triển khai ở chế độ chạy liên tục để luôn sẵn sàng lắng nghe các kết nối từ client. Đồng thời, server hỗ trợ xử lý đồng thời nhiều luồng (multi-threading) hoặc các cơ chế tương đương nhằm đáp ứng

hiều kết nối cùng lúc, bảo đảm thời gian phản hồi nhanh và khả năng mở rộng khi số lượng người dùng tăng lên.

2.2. Địa chỉ IP, localhost và port

Địa chỉ localhost thường tương ứng với 127.0.0.1, tức chính máy đang chạy chương trình. Port là số cổng giúp phân biệt các dịch vụ khác nhau trên cùng một máy. Ví dụ, web server có thể dùng cổng 80 hoặc 8080, còn MiniChat dùng cổng 9999.

Nếu client kết nối đến sai cổng hoặc server chưa mở cổng, hệ điều hành sẽ từ chối kết nối. Đó là nguyên nhân của lỗi Connection refused. Vì vậy, thứ tự chạy đúng là bật ChatServer trước rồi mới chạy ChatClient hoặc ChatClientGUI.

2.3. Giao thức TCP

TCP là giao thức hướng kết nối. Trước khi truyền dữ liệu, client và server phải thiết lập kết nối. Sau khi kết nối được tạo, dữ liệu được truyền theo dòng ổn định, có thứ tự và có cơ chế kiểm soát lỗi ở tầng giao thức.

Ứng dụng chat phù hợp với TCP vì tin nhắn cần được nhận đúng thứ tự. Nếu người dùng gửi 'Xin chào' rồi gửi tiếp 'Bạn khỏe không?', người nhận nên thấy hai tin theo đúng thứ tự đó.

2.4. Socket và ServerSocket trong Java

ServerSocket là lớp dùng phía server để mở một cổng và chờ client kết nối. Phương thức accept() sẽ dừng lại cho đến khi có client mới. Khi client kết nối thành công, accept() trả về một Socket đại diện cho kết nối cụ thể với client đó.

Socket là lớp dùng để trao đổi dữ liệu hai chiều. Từ Socket có thể lấy InputStream để đọc dữ liệu và OutputStream để ghi dữ liệu. Trong MiniChat, InputStream được bọc bằng BufferedReader, OutputStream được bọc bằng PrintWriter để đọc ghi theo từng dòng văn bản.

2.5. Luồng vào ra văn bản

BufferedReader hỗ trợ phương thức `readLine()`, rất phù hợp với ứng dụng chat văn bản vì mỗi tin nhắn có thể xem như một dòng. `PrintWriter` với chế độ `autoFlush` giúp gửi dữ liệu ngay khi gọi `println()`.

Việc dùng `println()` và `readLine()` tạo ra quy ước đơn giản: mỗi tin nhắn kết thúc bằng ký tự xuống dòng. Cách này dễ hiểu, dễ kiểm thử và phù hợp với đề tài học tập.

2.6. Lập trình đa luồng

Một server chat phải xử lý nhiều client đồng thời. Nếu chỉ dùng một luồng, server có thể bị kẹt khi đang đọc dữ liệu từ một client và không thể tiếp nhận client khác. Do đó, mỗi client cần có một luồng xử lý riêng.

Trong MiniChat, `ClientHandler` implements `Runnable`. Khi server nhận client mới, nó tạo `Thread thread = new Thread(clientHandler)` rồi gọi `thread.start()`. Mỗi `Thread` sẽ đọc tin nhắn từ client tương ứng và gọi server để phát tin nhắn.

2.7. Đồng bộ dữ liệu và danh sách client

Server lưu danh sách client bằng `ArrayList`. Danh sách này được dùng khi broadcast tin nhắn. Với quy mô đề tài nhỏ và chạy thử ít client, cách này đủ đơn giản. Tuy nhiên, trong hệ thống lớn, danh sách client nên được bảo vệ bằng cơ chế đồng bộ hoặc dùng cấu trúc dữ liệu `thread-safe`.

Nếu nhiều client cùng rời phòng hoặc gửi tin nhắn cùng lúc, việc đọc ghi danh sách clients có thể phát sinh rủi ro cạnh tranh dữ liệu. Đây là một điểm có thể cải tiến trong hướng phát triển.

2.8. Java Swing và giao diện người dùng

Swing là bộ thư viện giao diện truyền thống của Java. Các thành phần như JFrame, JPanel, JButton, JTextField, JTextArea và JLabel cho phép tạo cửa sổ ứng dụng desktop mà không cần cài thêm thư viện ngoài.

Một điểm quan trọng khi dùng Swing là cập nhật giao diện nên thực hiện trên Event Dispatch Thread. Trong đề tài, phương thức SwingUtilities.invokeLater() được dùng khi appendMessage để bảo đảm việc cập nhật vùng chat diễn ra an toàn với giao diện.

CHƯƠNG 3: NỘI DUNG THỰC HIỆN

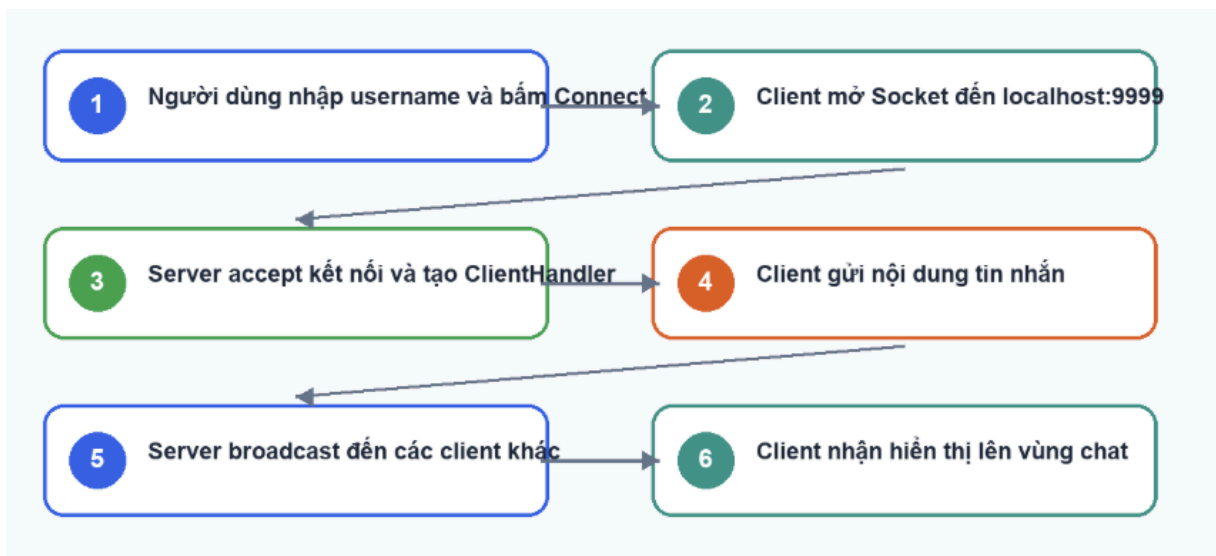
3.1. Thiết kế hệ thống

Thiết kế hệ thống được xây dựng theo hướng đơn giản, phù hợp với phạm vi đề tài. Server là trung tâm, client chỉ cần biết địa chỉ server và cổng kết nối. Cách thiết kế này giúp việc mở rộng thêm client mới rất dễ dàng.

3.1.1. Thiết kế kiến trúc

Server duy trì danh sách các ClientHandler đang hoạt động. Khi một client gửi tin nhắn, ClientHandler của client đó gọi ChatServer.broadcast(). Hàm broadcast duyệt danh sách client và gửi tin nhắn đến các client khác người gửi.

3.1.2. Thiết kế luồng xử lý



Hình 3. Luồng xử lý của dự án

Luồng xử lý bắt đầu từ thao tác kết nối. Sau khi server chấp nhận client, hai bên trao đổi dữ liệu qua socket. Khi người dùng gửi tin nhắn, nội dung đi từ giao diện client đến server, sau đó server chuyển tiếp đến các client còn lại.

3.1.3. Thiết kế lớp

Lớp	Thuộc tính/đối tượng chính	Phương thức chính
ChatServer	PORT, clients	main(), broadcast(), removeClient()
ClientHandler	socket, reader, writer, username	run(), sendMessage(), closeConnection()
ChatClientGUI	chatArea, usernameField, messageField, socket	connect(), receiveMessages(), sendMessage(), disconnect()
ChatClient	SERVER_HOST, SERVER_PORT	main(), parsePort()
ChatGroup	groupName, members	addMember(), hasMember()

Bảng 3. Danh sách lớp của đề tài

3.1.4. Thiết kế giao diện

Giao diện được chia thành ba vùng. Vùng trên chứa username và nút Connect/Disconnect. Vùng giữa là JTextArea hiển thị tin nhắn. Vùng dưới chứa trạng thái kết nối, ô nhập tin nhắn và nút Send. Bố cục này giúp người dùng thấy rõ mình đang kết nối hay chưa, nội dung chat đang có gì và thao tác gửi tin nhắn ở đâu.

3.2. Nội dung thực hiện

3.2.1. Cấu hình ChatServer

ChatServer là lớp khởi động phía server. Hằng PORT được đặt là 9999. Trong main(), chương trình tạo ServerSocket và sau đó đi vào vòng lặp vô hạn để chờ client. Mỗi khi accept() trả về một Socket, server tạo ClientHandler, thêm vào danh sách clients và khởi động luồng xử lý.

Phương thức broadcast() là điểm trung tâm của quá trình gửi tin nhắn. Nó duyệt qua danh sách client và gửi tin nhắn cho mọi client khác người gửi. Cách làm này tạo ra mô hình phòng chat chung: một người nói, những người còn lại nghe.

3.2.2. Cấu hình ClientHandler

ClientHandler đại diện cho một kết nối client cụ thể. Nó giữ socket, reader, writer và username. Khi run() được gọi, server yêu cầu client nhập username, sau đó liên tục đọc tin nhắn bằng readLine().

Nếu tin nhắn là /quit, vòng lặp kết thúc và kết nối được đóng. Nếu tin nhắn là nội dung thông thường, ClientHandler in ra console server và gọi ChatServer.broadcast() để gửi đến những client khác.

3.2.3. Cài đặt ChatClient dòng lệnh

ChatClient dòng lệnh giúp kiểm thử nhanh mà không cần giao diện. Client mở socket đến localhost:9999, tạo luồng riêng để nhận tin nhắn từ server và dùng luồng chính để đọc nội dung nhập từ bàn phím.

Sau khi sửa, ChatClient có thông báo lỗi rõ hơn khi không kết nối được. Nếu server chưa chạy, chương trình báo người dùng cần chạy ChatServer trước.

3.2.4. Cài đặt ChatClientGUI

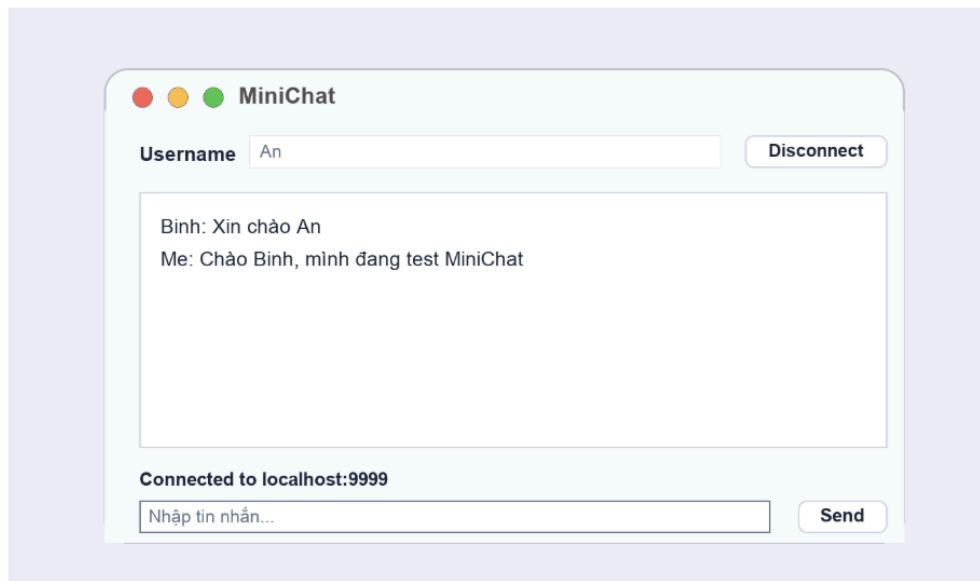
ChatClientGUI kế thừa JFrame. Giao diện được tạo trong buildLayout(), các sự kiện nút bấm được đăng ký trong bindActions(). Khi người dùng bấm Connect, phương thức connect() kiểm tra username, mở socket, tạo reader/writer, bật trạng thái kết nối và khởi động receiveThread.

receiveMessages() chạy trong một luồng riêng để giao diện không bị đứng khi chờ tin nhắn. Khi nhận được tin nhắn, chương trình gọi appendMessage(). Hàm này sử dụng SwingUtilities.invokeLater() để cập nhật JTextArea đúng luồng giao diện.

Để giao diện gọn hơn, client GUI không hiển thị dòng 'Connected to MiniChat Server.' trong vùng chat và bỏ qua dòng 'Enter your username:' do server gửi. Trạng thái kết nối được đặt ở statusLabel bên dưới.

3.2.5. Cập nhật module-info

Vì giao diện dùng Swing thuộc module java.desktop, file module-info.java cần khai báo requires java.desktop. Nếu thiếu khai báo này trong project dạng module, chương trình có thể không biên dịch hoặc không chạy được lớp giao diện.



Hình 4. Mô phỏng giao diện sau khi kết nối và gửi tin nhắn

3.2.6. Một số đoạn xử lý quan trọng

Server tạo ClientHandler cho mỗi client

```
Socket socket = serverSocket.accept();
ClientHandler clientHandler = new ClientHandler(socket);
clients.add(clientHandler);
Thread thread = new Thread(clientHandler);
thread.start();
```

Broadcast tin nhắn đến các client khác

```
public static void broadcast(String message, ClientHandler sender) {
    for (ClientHandler client : clients) {
        if (client != sender) {
            client.sendMessage(message);
        }
    }
}
```

Client GUI gửi tin nhắn

```
String message = messageField.getText().trim();
if (!connected || message.isEmpty()) {
    return;
}
serverWriter.println(message);
appendMessage("Me: " + message);
messageField.setText("");
```


CHƯƠNG 4: ĐÁNH GIÁ KẾT QUẢ

Kiểm thử được thực hiện theo các kịch bản thường gặp khi sử dụng ứng dụng chat. Các kịch bản bao gồm chạy server, chạy một client, chạy nhiều client, gửi tin nhắn, thoát client và kiểm tra lỗi khi server chưa chạy.

Mã test	Kịch bản	Kết quả mong đợi	Đánh giá
TC01	Chạy ChatServer	Console hiển thị server chạy tại cổng 9999.	Đạt.
TC02	Chạy client khi server chưa chạy	Client báo không kết nối được và yêu cầu chạy server trước.	Đạt.
TC03	Client GUI kết nối với username hợp lệ	Trạng thái chuyển sang Connected.	Đạt.
TC04	Client gửi tin nhắn	Trạng thái chuyển sang Connected.	Đạt.
TC05	Client nhập /quit	Client ngắt kết nối và server xóa khỏi danh sách.	Đạt.
TC06	Mở nhiều client	Server tạo nhiều ClientHandler.	Đạt ở phạm vi kiểm thử nhỏ.

Bảng 4. Đánh giá kết quả đạt được

4.1. Đánh giá chức năng

- Chức năng kết nối hoạt động đúng khi server chạy trước.
- Chức năng gửi nhận tin nhắn đáp ứng mục tiêu phòng chat chung.
- Giao diện Swing giúp người dùng thao tác dễ hơn so với terminal.
- Thông báo lỗi kết nối rõ ràng hơn sau khi xử lý ConnectException.
- Chương trình có thể chạy nhiều cửa sổ client để mô phỏng nhiều người dùng.

4.2. Đánh giá chức năng

Về kỹ thuật, đề tài thể hiện đúng mô hình socket TCP và đa luồng cơ bản. Code được chia thành các lớp tương đối rõ ràng. Tuy nhiên, danh sách clients hiện dùng ArrayList thông thường nên chưa tối ưu cho môi trường nhiều luồng lớn. Ngoài ra, giao thức tin nhắn còn đơn giản, chưa có định dạng dữ liệu chuẩn như JSON hoặc cơ chế phân loại tin nhắn.

4.3. Đánh giá trải nghiệm người dùng

Giao diện hiện tại đã có các thành phần cơ bản: tên người dùng, nút kết nối, vùng chat, trạng thái và nút gửi. Việc loại bỏ các dòng kỹ thuật thừa giúp vùng chat sạch hơn. Tuy vậy, giao diện vẫn có thể nâng cấp thêm danh sách người online, màu sắc phân biệt người gửi và người nhận, thời gian gửi tin nhắn và thanh cuộn đẹp hơn.

CHƯƠNG 5: KẾT LUẬN

5.1. Về vòng đời của server

Server trong MiniChat có vòng đời khác với chương trình thông thường. Sau khi khởi động, server không thực hiện một tác vụ rồi kết thúc ngay, mà duy trì trạng thái lắng nghe trong suốt thời gian ứng dụng hoạt động. Điều này thể hiện qua vòng lặp `while(true)` trong `ChatServer`. Vòng lặp này giúp server luôn sẵn sàng chấp nhận client mới, kể cả khi đã có nhiều client đang chat.

Khi server tạo `ServerSocket` tại cổng 9999, hệ điều hành cấp quyền cho chương trình nhận kết nối đi vào cổng này. Nếu cổng đã bị chương trình khác sử dụng, server sẽ không mở được và phải báo lỗi. Vì vậy, vị trí in thông báo 'MiniChat Server is running on port 9999' nên đặt sau khi `ServerSocket` tạo thành công, để tránh gây hiểu nhầm rằng server đã chạy trong khi thực tế cổng chưa mở.

Vòng đời server gồm bốn giai đoạn: khởi tạo, lắng nghe, phục vụ client và kết thúc. Trong bản hiện tại, server kết thúc khi người dùng dừng chương trình từ môi trường chạy. Ở phiên bản nâng cao, có thể thêm lệnh shutdown hoặc giao diện quản trị để server đóng tài nguyên chủ động hơn.

5.2. Về vòng đời của client

Client bắt đầu ở trạng thái chưa kết nối. Với `ChatClientGUI`, trạng thái này thể hiện bằng `statusLabel` hiển thị `Disconnected`, ô nhập tin nhắn bị vô hiệu hóa và nút gửi không dùng được. Cách này tránh việc người dùng nhập tin nhắn khi chưa có kết nối, giúp giảm lỗi sử dụng.

Khi người dùng nhập username và bấm `Connect`, client chuyển sang giai đoạn thiết lập kết nối. Nếu socket tạo thành công, client bật trạng thái `connected`, đổi nút `Connect` thành `Disconnect`, mở ô nhập tin nhắn và khởi động luồng nhận

dữ liệu. Nếu socket thất bại, giao diện hiển thị thông báo lỗi và quay về trạng thái ngắt kết nối.

Khi người dùng thoát bằng /quit hoặc nút Disconnect, client gửi thông báo thoát đến server, đóng reader, writer và socket. Việc đóng tài nguyên là bước quan trọng vì nếu bỏ qua, kết nối có thể tồn tại ở trạng thái treo, gây khó theo dõi khi kiểm thử nhiều lần.

5.3. Lựa chọn TCP thay vì UDP

Ứng dụng chat văn bản cần độ tin cậy cao hơn tốc độ tuyệt đối. Người nhận cần thấy đủ nội dung và đúng thứ tự. TCP phù hợp với yêu cầu này vì nó là giao thức hướng kết nối, tự đảm bảo thứ tự gói tin và phát hiện mất mát ở tầng giao thức.

Nếu dùng UDP, chương trình sẽ nhẹ hơn ở một số tình huống nhưng phải tự xử lý nhiều vấn đề như mất gói, sai thứ tự, gửi lại và xác nhận. Với phạm vi học tập của MiniChat, việc dùng TCP giúp tập trung vào logic client-server thay vì phải tự xây dựng cơ chế tin cậy.

Trong các ứng dụng chat thực tế, TCP hoặc các giao thức xây trên TCP như WebSocket thường được dùng cho tin nhắn văn bản. Điều này cho thấy lựa chọn TCP trong MiniChat không chỉ đơn giản mà còn phù hợp với bản chất bài toán.

5.4. Về cơ chế broadcast

Broadcast là cơ chế server gửi một tin nhắn đến nhiều client. Trong MiniChat, khi một ClientHandler nhận được tin nhắn, nó gọi ChatServer.broadcast(message, this). Tham số this đại diện cho người gửi, giúp server không gửi lại tin nhắn đó cho chính client đã gửi.

Cách không gửi lại cho người gửi có ưu điểm là tránh lặp nội dung. Ở ChatClientGUI, client đã tự hiển thị dòng 'Me: nội dung' ngay khi bấm Send.

Nếu server gửi lại cùng tin nhắn, vùng chat sẽ xuất hiện hai dòng giống nhau hoặc gây nhầm lẫn.

Trong phiên bản nâng cao, broadcast có thể được thay bằng routing. Khi đó, tin nhắn có trường receiver hoặc groupId. Server kiểm tra trường này để quyết định gửi cho một người, một nhóm hoặc toàn bộ phòng chat.

5.5. Về cách đặt username

Username hiện được nhập tự do từ client. Server chỉ đọc một dòng đầu tiên sau khi client kết nối và xem đó là tên người dùng. Cách này đơn giản, dễ hiểu và phù hợp với đề tài cơ bản.

Tuy nhiên, cách này chưa kiểm tra trùng tên, chưa chặn tên rỗng ở phía server và chưa xác thực người dùng. ChatClientGUI đã kiểm tra username rỗng trước khi kết nối, nhưng server vẫn nên có kiểm tra riêng vì không thể tin hoàn toàn vào client.

Khi mở rộng, server nên duy trì danh sách username đang online. Nếu tên đã tồn tại, server gửi thông báo yêu cầu client nhập tên khác. Nếu có hệ thống tài khoản, username sẽ đến từ quá trình đăng nhập thay vì nhập tự do mỗi lần mở ứng dụng.

5.6. Xử lý lỗi Connection refused

Connection refused là lỗi rất thường gặp khi học socket. Lỗi này không có nghĩa code client sai hoàn toàn, mà thường nghĩa là không có chương trình nào đang lắng nghe tại địa chỉ và cổng mà client muốn kết nối.

Trong MiniChat, client kết nối đến localhost:9999. Nếu ChatServer chưa chạy, cổng 9999 chưa mở nên hệ điều hành từ chối kết nối. Vì vậy, cách xử lý đúng là chạy server trước, quan sát thông báo server đang chạy rồi mới mở client.

Việc sửa ChatClient để bắt ConnectException giúp người dùng hiểu nguyên nhân nhanh hơn. Thay vì chỉ thấy 'Client error: Connection refused', chương trình báo rõ không thể kết nối đến server và yêu cầu chạy ChatServer trước.

5.7. Lỗi module trong Java

Project hiện có module-info.java, nghĩa là chương trình chạy theo cơ chế Java module. Khi chạy bằng lệnh `java -p bin -m Minichat/minichat.ChatClientGUI`, Java sẽ tìm module Minichat trong module path bin.

Nếu chưa biên dịch, biên dịch sai thư mục hoặc đang đứng sai thư mục khi chạy lệnh, Java sẽ báo Module Minichat not found. Đây là lỗi liên quan đến cách tổ chức đầu ra biên dịch, không phải lỗi socket.

Cách xử lý là bảo đảm lệnh `javac -d bin src/module-info.java src/minichat/*.java` đã chạy thành công và trong bin có module-info.class cùng thư mục minichat. Khi dùng Eclipse, người học có thể chạy trực tiếp Run As > Java Application để tránh nhầm lẫn lệnh terminal.

5.8. Cập nhật giao diện Swing

Project hiện có module-info.java, nghĩa là chương trình chạy theo cơ chế Java module. Khi chạy bằng lệnh `java -p bin -m Minichat/minichat.ChatClientGUI`, Java sẽ tìm module Minichat trong module path bin.

Trong ChatClientGUI, luồng receiveThread dùng để đọc tin nhắn từ server. Khi nhận được tin nhắn, nó không trực tiếp chỉnh sửa JTextArea theo cách thô, mà gọi appendMessage. Bên trong appendMessage có SwingUtilities.invokeLater để đưa thao tác cập nhật giao diện về đúng luồng.

Đây là chi tiết kỹ thuật nhỏ nhưng quan trọng. Nó cho thấy khi kết hợp lập trình mạng và giao diện, người lập trình phải quan tâm cả luồng nhận dữ liệu lẫn luồng hiển thị giao diện.

5.9. Giao thức tin nhắn

Danh sách clients trong ChatServer hiện là ArrayList. Danh sách này lưu các ClientHandler đang kết nối và được dùng mỗi khi broadcast. Với bài tập nhỏ, ArrayList dễ dùng và dễ hiểu.

Tuy nhiên, ArrayList không phải cấu trúc thread-safe. Nếu một luồng đang duyệt danh sách để broadcast trong khi luồng khác xóa client khỏi danh sách, chương trình có thể phát sinh lỗi hoặc hành vi không ổn định.

Giải pháp nâng cao là dùng CopyOnWriteArrayList hoặc đồng bộ hóa các thao tác thêm, xóa và duyệt danh sách. Với ứng dụng chat nhiều người, đây là cải tiến quan trọng để tăng độ ổn định.

5.10. Quản lý danh sách client

Bản hiện tại gửi tin nhắn dưới dạng chuỗi văn bản đơn giản. Ví dụ, khi An gửi 'Xin chào', server tạo chuỗi 'An: Xin chào' và gửi đến client khác. Cách này dễ hiểu, nhưng khó mở rộng khi có nhiều loại thông điệp.

Nếu muốn thêm chat riêng, chat nhóm, thông báo online, thông báo đang nhập hoặc gửi file, chuỗi văn bản đơn giản sẽ không đủ rõ. Khi đó nên định nghĩa cấu trúc Message gồm type, sender, receiver, group, content và timestamp.

Một lựa chọn phổ biến là dùng JSON. Client gửi JSON đến server, server phân tích trường type và xử lý theo từng loại. Cách này giúp giao thức rõ ràng hơn, dễ debug và dễ phát triển thêm chức năng.

5.11. Phân tích bảo mật

MiniChat hiện phù hợp môi trường học tập và localhost. Dữ liệu truyền qua socket là văn bản thuần, nghĩa là nếu chạy trong mạng không an toàn, người khác có thể nghe lén nội dung nếu có quyền truy cập mạng phù hợp.

Ứng dụng cũng chưa có xác thực. Bất kỳ client nào kết nối được đến cổng server đều có thể nhập username và tham gia. Điều này là chấp nhận được cho đề tài cơ bản, nhưng không đủ cho sản phẩm thực tế.

Để cải thiện bảo mật, có thể thêm đăng nhập, mã hóa kết nối bằng SSL/TLS, giới hạn địa chỉ IP, kiểm tra nội dung đầu vào và ghi log sự kiện quan trọng. Bảo mật nên được thiết kế từ đầu nếu ứng dụng dùng ngoài phạm vi học tập.

5.12. Khả năng lưu trữ dữ liệu

Hiện tại, khi server tắt, toàn bộ lịch sử chat biến mất. Đây là đặc điểm của ứng dụng chưa có tầng lưu trữ. Với yêu cầu học socket, điều này giúp chương trình gọn nhẹ.

Nếu muốn lưu lịch sử, có thể bắt đầu bằng cách ghi tin nhắn vào file text. Mỗi dòng gồm thời gian, người gửi và nội dung. Cách này đơn giản nhưng khó truy vấn khi dữ liệu lớn.

Giải pháp tốt hơn là dùng cơ sở dữ liệu như SQLite hoặc MySQL. Khi đó, mỗi tin nhắn là một bản ghi có id, sender, receiver hoặc groupId, content và createdAt. Giao diện có thể tải lại lịch sử khi người dùng đăng nhập.

5.13. Chức năng chat nhóm

Project hiện có lớp ChatGroup với groupName và danh sách members. Điều này cho thấy hướng phát triển chat nhóm đã được chuẩn bị sơ bộ, dù logic server hiện chưa sử dụng lớp này.

Để hoàn thiện chat nhóm, server cần quản lý nhiều ChatGroup. Khi người dùng tạo nhóm, server tạo đối tượng ChatGroup mới. Khi người dùng tham gia nhóm, username được thêm vào members. Khi gửi tin nhắn nhóm, server chỉ gửi đến các client có username thuộc nhóm đó.

Chat nhóm cũng cần giao thức rõ ràng. Ví dụ, client gửi lệnh /create tenNhom, /join tenNhom hoặc Message type GROUP_MESSAGE. Nếu tiếp tục dùng chuỗi tự do, việc phân tích lệnh sẽ dễ sai khi nội dung tin nhắn phức tạp.

5.14. Chức năng chat riêng

Chat riêng là chức năng cho phép một người gửi tin nhắn đến một người cụ thể. Khác với broadcast, server phải tìm đúng ClientHandler của người nhận. Điều này yêu cầu server có ánh xạ từ username sang ClientHandler.

Có thể dùng Map<String, ClientHandler> onlineUsers để lưu người dùng đang online. Khi client đăng nhập username, server thêm vào map. Khi client rời phòng, server xóa khỏi map. Khi gửi tin nhắn riêng, server tra map để tìm người nhận.

Giao diện cũng cần thay đổi. Thay vì chỉ có một phòng chat chung, client nên có danh sách người online hoặc ô nhập tên người nhận. Đây là phần mở rộng hợp lý sau khi bản chat chung đã ổn định.

5.15. Đóng gói và triển khai

Trong quá trình học, chạy bằng javac và java là phù hợp vì người học thấy rõ quá trình biên dịch và chạy module. Tuy nhiên, với người dùng cuối, cách chạy này chưa thuận tiện.

Ứng dụng có thể được đóng gói thành file JAR. Khi đó, người dùng chỉ cần chạy một lệnh đơn giản hơn hoặc bấm đúp nếu hệ điều hành đã liên kết Java đúng cách. Với ứng dụng desktop, có thể dùng jpackage để tạo gói cài đặt.

Khi triển khai qua mạng LAN, server nên chạy trên một máy cố định. Client thay localhost bằng địa chỉ IP của máy server. Đồng thời, tường lửa cần cho phép kết nối đến cổng 9999.

5.16. Kiểm thử mở rộng

Kiểm thử thủ công hiện đủ để xác nhận chức năng cơ bản. Tuy nhiên, khi chương trình lớn hơn, kiểm thử thủ công dễ bỏ sót lỗi. Nên bổ sung kiểm thử tự động cho các hàm xử lý giao thức và quản lý danh sách client.

Với socket, kiểm thử tự động có thể phức tạp hơn vì cần khởi động server trong test, mở client giả lập và kiểm tra dữ liệu nhận được. Dù vậy, đây là hướng rất hữu ích nếu muốn nâng chất lượng project.

Ngoài kiểm thử chức năng, cần kiểm thử tải nhẹ: mở nhiều client, gửi tin nhắn liên tục, ngắt kết nối đột ngột và chạy lại server nhiều lần. Những kịch bản này giúp phát hiện lỗi tài nguyên và lỗi đa luồng.

5.17. Khả năng bảo trì mã nguồn

Một chương trình dễ bảo trì là chương trình mà người khác có thể đọc, hiểu và sửa mà không phải đoán quá nhiều. Trong MiniChat, việc chia thành ChatServer, ClientHandler, ChatClient và ChatClientGUI giúp mỗi lớp có trách nhiệm tương đối rõ ràng. Server quản lý kết nối, ClientHandler xử lý một client, còn giao diện nằm riêng trong lớp GUI.

Tuy vậy, khi chương trình phát triển thêm, cần tiếp tục tách lớp. Ví dụ, logic giao thức nên đưa vào lớp riêng thay vì đặt trực tiếp trong giao diện. Logic quản lý người dùng nên tách khỏi ChatServer. Việc tách này giúp mỗi lớp nhỏ hơn, dễ kiểm thử hơn và hạn chế sửa một chức năng làm ảnh hưởng chức năng khác.

Bảo trì cũng liên quan đến đặt tên biến, xử lý lỗi và ghi chú vừa đủ. Các tên như serverReader, serverWriter, messageField, connectButton thể hiện đúng

ý nghĩa. Nếu thêm chức năng phức tạp, nên bổ sung comment ở những đoạn xử lý đa luồng hoặc giao thức để người đọc hiểu nhanh mục đích.

5.18. Trải nghiệm khi dùng Eclipse

Eclipse là môi trường phổ biến khi học Java, nhưng với project có server và client, người mới dễ nhầm vì cần chạy nhiều chương trình cùng lúc. Nếu chỉ chạy ChatClientGUI mà chưa chạy ChatServer, client sẽ báo không kết nối được. Điều này là đúng về mặt kỹ thuật nhưng ban đầu có thể gây khó hiểu.

Cách thao tác phù hợp trong Eclipse là chạy ChatServer.java trước bằng Run As > Java Application. Sau đó mở ChatClientGUI.java và chạy tiếp bằng Run As > Java Application. Muốn kiểm thử nhiều người dùng thì chạy ChatClientGUI thêm một hoặc nhiều lần nữa. Mỗi lần chạy sẽ mở một cửa sổ client riêng.

Khi viết báo cáo hoặc hướng dẫn sử dụng, cần nhấn mạnh thứ tự chạy. Server giống như phòng họp phải được mở trước. Client giống như người tham gia, chỉ có thể vào phòng khi phòng đã tồn tại. Cách giải thích này giúp người dùng hiểu bản chất thay vì chỉ ghi nhớ lệnh.

5.19. Trải nghiệm khi dùng Eclipse

Dữ liệu vào của server là các dòng văn bản nhận từ client. Dòng đầu tiên sau khi kết nối được xem là username. Các dòng tiếp theo được xem là tin nhắn, trừ khi dòng đó là /quit. Dữ liệu ra của server là thông báo gửi đến các client khác, thường có dạng 'username: nội dung'.

Dữ liệu vào của client GUI gồm username và nội dung tin nhắn do người dùng nhập. Dữ liệu ra của client là nội dung hiển thị trong JTextArea và trạng thái kết nối ở JLabel. Việc phân biệt dữ liệu vào/ra giúp dễ thiết kế kiểm thử và dễ tìm lỗi khi chương trình không hoạt động như mong đợi.

Trong phiên bản nâng cao, dữ liệu vào nên được kiểm tra kỹ hơn. Ví dụ, username quá dài, tin nhắn rỗng, tin nhắn chứa ký tự đặc biệt hoặc nội dung quá lớn đều cần quy tắc xử lý. Đây là bước quan trọng nếu ứng dụng chuyển từ bài tập sang sản phẩm sử dụng thực tế.

5.20. Hiệu năng ở quy mô nhỏ

Ở quy mô vài client, cách triển khai hiện tại hoạt động tốt vì mỗi client có một Thread riêng và tin nhắn là văn bản ngắn. Server không cần thuật toán phức tạp, chỉ cần duyệt danh sách client và gửi nội dung đến từng người.

Tuy nhiên, khi số client tăng lên hàng trăm hoặc hàng nghìn, mô hình một Thread cho một client có thể tiêu tốn nhiều tài nguyên. Mỗi Thread cần bộ nhớ stack và hệ điều hành phải chuyển ngữ cảnh giữa nhiều luồng. Khi đó, server có thể chậm hoặc không ổn định.

5.21. Tính đúng đắn của thao tác thoát

Thoát kết nối là một phần quan trọng của ứng dụng mạng. Nếu client chỉ đóng cửa sổ mà không thông báo, server có thể chỉ phát hiện sau khi đọc socket thất bại. Nếu client gửi /quit trước khi đóng, server có thể xử lý rõ ràng hơn và thông báo cho các client khác.

Trong ChatClientGUI, khi người dùng bấm Disconnect, chương trình gửi /quit nếu đang kết nối, sau đó đóng tài nguyên. Khi người dùng nhập /quit vào ô chat, chương trình gửi lệnh này một lần rồi ngắt kết nối. Việc tránh gửi lặp giúp server không nhận hai lệnh thoát không cần thiết.

Ở server, finally trong ClientHandler bảo đảm closeConnection() được gọi dù vòng lặp kết thúc bình thường hay có IOException. Đây là cách viết tốt vì tài nguyên được giải phóng trong mọi tình huống chính.

5.22. Khả năng quốc tế hóa giao diện

Giao diện hiện dùng tiếng Anh cho các nhãn như Username, Connect, Disconnect và Send. Với người dùng Việt Nam, có thể đổi thành Tên người dùng, Kết nối, Ngắt kết nối và Gửi để thân thiện hơn.

Nếu muốn hỗ trợ nhiều ngôn ngữ, không nên viết trực tiếp chuỗi hiển thị trong code. Thay vào đó có thể dùng file cấu hình hoặc ResourceBundle. Khi đổi ngôn ngữ, chương trình chỉ cần tải bộ chuỗi tương ứng mà không phải sửa logic xử lý.

Dù đây là đề tài nhỏ, việc nghĩ đến quốc tế hóa giúp người học hiểu một nguyên tắc thiết kế phần mềm quan trọng: tách nội dung hiển thị khỏi logic nghiệp vụ. Điều này làm chương trình dễ điều chỉnh khi yêu cầu thay đổi.

5.23. Ghi log và quan sát hệ thống

Hiện server in thông tin ra console khi client kết nối, gửi tin nhắn hoặc rời phòng. Đây là hình thức log đơn giản nhưng hữu ích trong giai đoạn học tập. Nhờ console, người thực hiện biết server có nhận kết nối hay không và tin nhắn đi qua server như thế nào.

Trong ứng dụng thực tế, log nên có mức độ như INFO, WARN và ERROR. Log cũng nên ghi thời gian, tên luồng, địa chỉ client và nội dung sự kiện. Điều này giúp phân tích sự cố nhanh hơn khi có nhiều client hoạt động cùng lúc.

Tuy nhiên, cũng cần chú ý không ghi dữ liệu nhạy cảm quá nhiều vào log. Nếu ứng dụng có mật khẩu hoặc tin nhắn riêng tư, log phải được thiết kế cẩn thận để phục vụ debug mà không làm lộ thông tin người dùng.

5.24. Giá trị học tập của đề tài

MiniChat là đề tài nhỏ nhưng có tính tổng hợp cao. Người học không chỉ viết một hàm xử lý đơn lẻ mà phải kết nối nhiều khái niệm: cấu trúc project Java, module, socket, luồng vào ra, đa luồng, giao diện Swing và xử lý lỗi.

Khi gặp lỗi như Connection refused hoặc Module not found, người học có cơ hội hiểu cách chương trình tương tác với môi trường chạy. Đây là trải nghiệm quan trọng vì phát triển phần mềm thực tế không chỉ là viết code đúng mà còn là cấu hình, chạy, kiểm thử và sửa lỗi.

Sau khi hoàn thành đề tài, người học có nền tảng để tiếp tục xây dựng các ứng dụng mạng khác như hệ thống gửi file, trò chơi nhiều người chơi đơn giản, ứng dụng thông báo nội bộ hoặc hệ thống hỗ trợ khách hàng trong mạng LAN.

5.25. Kết luận tổng thể

Đề tài MiniChat đã hoàn thành mục tiêu xây dựng một ứng dụng chat cơ bản bằng Java. Chương trình thể hiện được cách server mở cổng, client kết nối đến server, hai bên trao đổi dữ liệu văn bản và server xử lý nhiều client bằng đa luồng.

Việc bổ sung giao diện Swing làm cho đề tài trực quan hơn. Người dùng không cần thao tác hoàn toàn bằng terminal mà có thể nhập username, gửi tin nhắn và quan sát trạng thái kết nối trên cửa sổ ứng dụng.

Thông qua đề tài, người thực hiện hiểu rõ hơn về TCP socket, ServerSocket, Socket, BufferedReader, PrintWriter, Thread và nguyên tắc cập nhật giao diện Swing. Đây là nền tảng quan trọng để tiếp tục phát triển các ứng dụng mạng phức tạp hơn.

DANH MỤC TÀI LIỆU THAM KHẢO

1. Java Platform, Standard Edition Documentation - `java.net.ServerSocket`.
2. Java I/O Documentation - `BufferedReader`, `InputStreamReader` và `PrintWriter`.
3. Think Java – David Allen, Tiếng Việt, 2022 (mindx.edu.vn).
4. Java Network Programming – Elliotte Rusty Harold, 2013.
5. Network Programming with Java – Jan Graba, 2002.
6. Free Java Guide – <https://freejavaguide.com>.
7. Java Concurrency in Practice – Brian Goetz, 2006.
8. Optimizing Java – Benjamin Evans, 2018.
9. Mã nguồn project MiniChat, package `src/minichat`.

PHỤ LỤC

Phụ lục A. Hướng dẫn chạy chương trình

* Code ChatClient.java

```
package minichat;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ConnectException;
import java.net.Socket;

public class ChatClient {
    private static final String DEFAULT_SERVER_HOST = "localhost";
    private static final int SERVER_PORT = 9999;

    public static void main(String[] args) {
        String serverHost = args.length > 0 ? args[0] :
        DEFAULT_SERVER_HOST;
        int serverPort = args.length > 1 ? parsePort(args[1]) : SERVER_PORT;

        try (
            Socket socket = new Socket(serverHost, serverPort);
            BufferedReader serverReader = new BufferedReader(new
            InputStreamReader(socket.getInputStream()));
            PrintWriter serverWriter = new PrintWriter(socket.getOutputStream(),
            true);
            BufferedReader keyboardReader = new BufferedReader(new
            InputStreamReader(System.in))
        ) {
            System.out.println("Connected to MiniChat Server.");

            Thread receiveThread = new Thread(() -> {
                try {
                    String serverMessage;

                    while ((serverMessage = serverReader.readLine()) != null) {
                        System.out.println(serverMessage);
                    }
                } catch (IOException e) {
                    e.printStackTrace();
                }
            });
            receiveThread.start();
        } catch (ConnectException e) {
            System.out.println("Cannot connect to the server. Please check the host and port.");
        }
    }
}
```



```
        }

        } catch (IOException e) {
            System.out.println("Disconnected from server.");
        }
    });

    receiveThread.start();

    String userInput;

    while ((userInput = keyboardReader.readLine()) != null) {
        serverWriter.println(userInput);

        if (userInput.equalsIgnoreCase("/quit")) {
            break;
        }
    }

    } catch (IOException e) {
        if (e instanceof ConnectException) {
            System.out.println("Cannot connect to MiniChat Server at " +
serverHost + ":" + serverPort + ".");
            System.out.println("Please run ChatServer first, then run ChatClient
again.");
        } else {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}

private static int parsePort(String value) {
    try {
        return Integer.parseInt(value);
    } catch (NumberFormatException e) {
        System.out.println("Invalid port '" + value + "'. Using default port " +
SERVER_PORT + ".");
        return SERVER_PORT;
    }
}
}
```

*** Code ChatGroup.java**

```
package minichat;

import java.util.ArrayList;
import java.util.List;

public class ChatGroup {
    private String groupName;
    private List<String> members;

    public ChatGroup(String groupName) {
        this.groupName = groupName;
        this.members = new ArrayList<>();
    }

    public String getGroupName() {
        return groupName;
    }

    public List<String> getMembers() {
        return members;
    }

    public void addMember(String username) {
        if (!members.contains(username)) {
            members.add(username);
        }
    }

    public boolean hasMember(String username) {
        return members.contains(username);
    }
}
```

*** Code ChatServer.java**

```
package minichat;

import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.ArrayList;
import java.util.List;

public class ChatServer {
    private static final int PORT = 9999;
    private static final List<ClientHandler> clients = new ArrayList<>();

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("MiniChat Server is running on port " + PORT);

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected: " + socket);

                ClientHandler clientHandler = new ClientHandler(socket);
                clients.add(clientHandler);

                Thread thread = new Thread(clientHandler);
                thread.start();
            }

        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }

    public static void broadcast(String message, ClientHandler sender) {
        for (ClientHandler client : clients) {
            if (client != sender) {
                client.sendMessage(message);
            }
        }
    }
}
```

```
    }  
}  
  
public static void removeClient(ClientHandler clientHandler) {  
    clients.remove(clientHandler);  
    System.out.println("A client disconnected.");  
}  
}
```

*** Code ClientHandler.java**

```
package minichat;  
  
import java.io.BufferedReader;  
import java.io.IOException;  
import java.io.InputStreamReader;  
import java.io.PrintWriter;  
import java.net.Socket;  
  
public class ClientHandler implements Runnable {  
    private Socket socket;  
    private BufferedReader reader;  
    private PrintWriter writer;  
    private String username;  
  
    public ClientHandler(Socket socket) {  
        this.socket = socket;  
  
        try {  
            reader = new BufferedReader(new  
InputStreamReader(socket.getInputStream()));  
            writer = new PrintWriter(socket.getOutputStream(), true);  
        } catch (IOException e) {  
            System.out.println("ClientHandler error: " + e.getMessage());  
        }  
    }  
  
    @Override  
    public void run() {  
        try {
```

```
writer.println("Enter your username:");
username = reader.readLine();

System.out.println(username + " joined the chat.");
ChatServer.broadcast(username + " joined the chat.", this);

String message;

while ((message = reader.readLine()) != null) {
    if (message.equalsIgnoreCase("/quit")) {
        break;
    }

    System.out.println(username + ": " + message);
    ChatServer.broadcast(username + ": " + message, this);
}

} catch (IOException e) {
    System.out.println("Connection error: " + e.getMessage());
} finally {
    closeConnection();
}
}

public void sendMessage(String message) {
    writer.println(message);
}

private void closeConnection() {
    try {
        ChatServer.removeClient(this);
        ChatServer.broadcast(username + " left the chat.", this);

        if (reader != null) {
            reader.close();
        }

        if (writer != null) {
            writer.close();
        }
    }
```

```
        if (socket != null) {  
            socket.close();  
        }  
  
    } catch (IOException e) {  
        System.out.println("Close connection error: " + e.getMessage());  
    }  
}  
}
```

*** Code MessageProtocol.java**

```
package minichat;  
public class MessageProtocol {  
    public static final String LOGIN = "/login";  
    public static final String CREATE_GROUP = "/create";  
    public static final String JOIN_GROUP = "/join";  
    public static final String GROUP_MESSAGE = "/group";  
    public static final String PRIVATE_MESSAGE = "/private";  
    public static final String LIST = "/list";  
    public static final String QUIT = "/quit";  
}
```

Phụ lục B. Hướng dẫn chạy chương trình

B.1. Chạy bằng Terminal

- Mở Terminal và chuyển đến thư mục project: `cd /Users/macOS/Desktop/Web_dev/Minichat/Minichat`
- Biên dịch: `javac -d bin src/module-info.java src/minichat/*.java`
- Chạy server: `java -p bin -m Minichat/minichat.ChatServer`
- Mở Terminal khác và chạy client giao diện: `java -p bin -m Minichat/minichat.ChatClientGUI`.
- Nhập username, bấm Connect, nhập tin nhắn và bấm Send.

B.2. Chạy bằng Terminal

- Mở project trong Eclipse.
- Mở file ChatServer.java, chọn Run As > Java Application.
- Giữ server đang chạy.
- Mở file ChatClientGUI.java, chọn Run As > Java Application.
- Muốn test nhiều người dùng thì chạy ChatClientGUI thêm lần nữa.

Phụ lục C. Một số lỗi thường gặp

Lỗi	Nguyên nhân	Cách xử lý
Connection refused	Client chạy khi server chưa mở cổng 9999.	Chạy ChatServer trước rồi chạy client.
Module Minichat not found	Chưa biên dịch vào bin hoặc đang đứng sai thư mục.	Chưa biên dịch vào bin hoặc đang đứng sai thư mục.
Không thấy Terminal trong Eclipse	Eclipse chưa mở view Terminal.	Window > Show View > Other > Terminal.
Giao diện không mở	Thiếu requires java.desktop trong module-info.	Thêm requires java.desktop và biên dịch lại.
Không thấy tin nhắn ở client khác	Chỉ mở một client hoặc client khác chưa kết nối.	Mở ít nhất hai cửa sổ client.

Đường dẫn xem code github:

https://github.com/Dung2711-cmd/PTUDHDV_dungnq110695